

Server Monitoring tool

Nagios Server Monitoring

Description

1. Install and check the functioning of server monitoring tools like:
 - a. Nagios
 - b. Zabbix
2. Deeply study commands used to monitor the server as well as services.

Objective

The objective of this practical was to install and configure **Nagios Server Monitoring** on AWS EC2 instances and monitor both the Nagios server and a remote client server.

The practical included:

- Installing Nagios Core.
- Installing Nagios Plugins.
- Configuring Apache for the Nagios web interface.
- Configuring NRPE for remote monitoring.
- Adding a remote Linux server to Nagios.
- Monitoring CPU load, disk usage, logged-in users, processes, SSH and HTTP.
- Testing communication between the Nagios server and client.
- Monitoring CPU/system load on the remote client.
- Generating an intentional CPU load on the client.
- Verifying the Nagios status changes from **OK** → **CRITICAL** → **OK**.

AWS Infrastructure Used

Two RHEL EC2 instances were used.

Server	Purpose	Private IP	Operating System
servera	Nagios Server	172.31.5.105	RHEL 10.2
serverb	Nagios Client	172.31.4.76	RHEL 10.2

Server Roles

servera

- Nagios Core
- Nagios Plugins
- Apache
- Nagios Web Interface
- check_nrpe plugin

serverb

- NRPE
- Nagios Plugins
- Services monitored remotely by servera

Security Group Configuration

Both EC2 instances were configured using the same AWS Security Group.

The required communication was allowed between the two instances.

ICMP

An inbound rule was added:

Type: All ICMP - IPv4

Source: Same Security Group

This allowed servera and serverb to communicate using ICMP and allowed Nagios to perform the host availability check.

NRPE

TCP port **5666** was allowed between the Nagios server and client.

Port: 5666
Protocol: TCP
Purpose: NRPE communication

Nagios Server Installation

Update the System

Server: servera

The system packages were updated:

```
sudo dnf update -y
```

Install Required Dependencies

Server: servera

The packages required to compile and run Nagios were installed:

```
sudo dnf install -y gcc glibc glibc-common perl make gettext  
autoconf automake wget unzip httpd php gd gd-devel openssl-devel
```

Create Nagios User and Group

Server: servera

The Nagios user and command group were created:

```
sudo useradd nagios  
sudo groupadd nagcmd  
sudo usermod -aG nagcmd nagios  
sudo usermod -aG nagcmd apache
```

Download and Install Nagios Core

Server: servera

Nagios Core 4.5.9 was downloaded:

wget <https://assets.nagios.com/downloads/nagioscore/releases/nagios-4.5.9.tar.gz>

The archive was extracted and the Nagios source directory was entered.

The configuration was prepared:

```
./configure --with-command-group=nagcmd
```

Nagios was compiled:

```
make all
```

Nagios was installed:

```
sudo make install  
sudo make install-config  
sudo make install-daemoninit  
sudo make install-webconf  
sudo make install-commandmode
```

Configure Nagios Web Authentication

Server: servera

A Nagios web user named nagiosadmin was created:

```
sudo htpasswd -c /usr/local/nagios/etc/htpasswd.users nagiosadmin
```

A password was configured for the nagiosadmin account.

The password was not included in the documentation.

Install Nagios Plugins

Server: servera

Nagios Plugins version 2.4.12 was downloaded:

wget <https://nagios-plugins.org/download/nagios-plugins-2.4.12.tar.gz>

The archive was extracted and the source directory was entered.

The configuration was performed:

```
./configure --with-nagios-user=nagios --with-nagios-group=nagios
```

The plugins were compiled:

```
make
```

The plugins were installed:

```
sudo make install
```

Validate Nagios Configuration

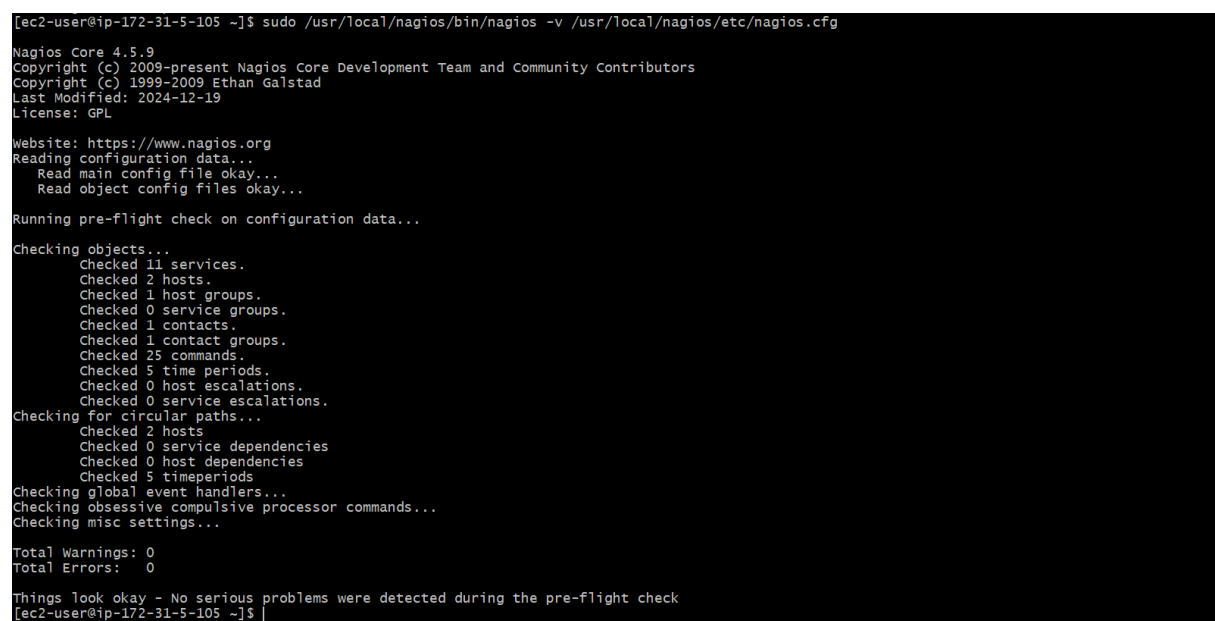
Server: servera

The Nagios configuration was checked for errors:

```
sudo /usr/local/nagios/bin/nagios -v  
/usr/local/nagios/etc/nagios.cfg
```

The configuration validation completed successfully.

Screenshot – Nagios Configuration Validation

A terminal window showing the output of the Nagios configuration validation command. The output displays the Nagios Core version (4.5.9), copyright information, and a detailed list of checked configuration items including services, hosts, groups, contacts, commands, time periods, escalations, and dependencies. It concludes with 'Total Warnings: 0', 'Total Errors: 0', and a confirmation that 'Things look okay - No serious problems were detected during the pre-flight check'.

```
[ec2-user@ip-172-31-5-105 ~]$ sudo /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg  
Nagios Core 4.5.9  
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors  
Copyright (c) 1999-2009 Ethan Galstad  
Last Modified: 2024-12-19  
License: GPL  
  
website: https://www.nagios.org  
Reading configuration data...  
  Read main config file okay...  
  Read object config files okay...  
  
Running pre-flight check on configuration data...  
  
Checking objects...  
  Checked 11 services.  
  Checked 2 hosts.  
  Checked 1 host groups.  
  Checked 0 service groups.  
  Checked 1 contacts.  
  Checked 1 contact groups.  
  Checked 25 commands.  
  Checked 5 time periods.  
  Checked 0 host escalations.  
  Checked 0 service escalations.  
Checking for circular paths...  
  Checked 2 hosts  
  Checked 0 service dependencies  
  Checked 0 host dependencies  
  Checked 5 timeperiods  
Checking global event handlers...  
Checking obsessive compulsive processor commands...  
Checking misc settings...  
  
Total Warnings: 0  
Total Errors: 0  
  
Things look okay - No serious problems were detected during the pre-flight check  
[ec2-user@ip-172-31-5-105 ~]$
```

Start and Verify Nagios

Server: servera

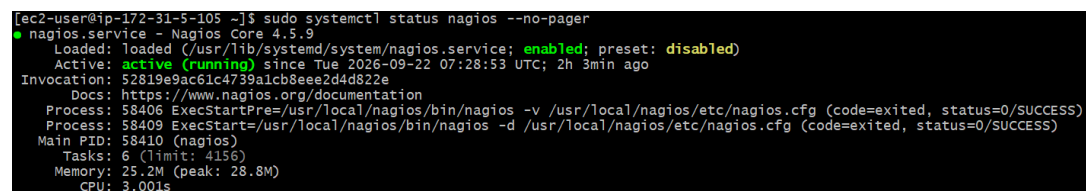
The Nagios service was started and enabled.

The service status was checked:

```
sudo systemctl status nagios --no-pager
```

The service was confirmed to be active and running.

Screenshot – Nagios Service



```
[ec2-user@ip-172-31-5-105 ~]$ sudo systemctl status nagios --no-pager
● nagios.service - Nagios Core 4.5.9
   Loaded: loaded (/usr/lib/systemd/system/nagios.service; enabled; preset: disabled)
   Active: active (running) since Tue 2025-09-22 07:28:53 UTC; 2h 3min ago
     Invocation: 52819e9ac61c4739a1cb8eee2d4d822e
       Docs: https://www.nagios.org/documentation
    Process: 58406 ExecStartPre=/usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg (code=exited, status=0/SUCCESS)
    Process: 58409 ExecStart=/usr/local/nagios/bin/nagios -d /usr/local/nagios/etc/nagios.cfg (code=exited, status=0/SUCCESS)
   Main PID: 58410 (nagios)
      Tasks: 6 (limit: 4156)
     Memory: 25.2M (peak: 28.8M)
        CPU: 3.001s
```

Access Nagios Web Interface

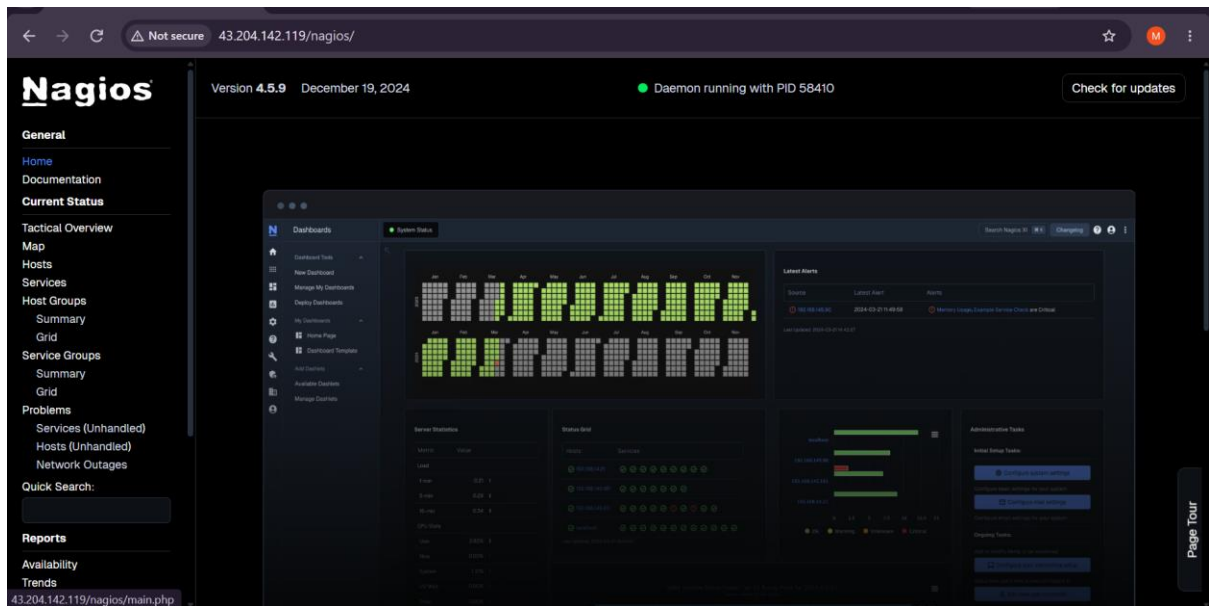
The Nagios web interface was opened using:

<http://<servera-public-IP>/nagios/>

The nagiosadmin account was used for authentication.

The Nagios dashboard displayed the monitoring information.

Screenshot – Nagios Dashboard



Server-Side Monitoring Commands

Several Nagios plugins were tested directly on servera to understand how individual monitoring checks work.

Disk Monitoring

Server: servera

The check_disk plugin was used to monitor disk space:

```
/usr/local/nagios/libexec/check_disk -w 20% -c 10%
```

The command returned a normal disk status.

Screenshot – Disk Monitoring

```
[ec2-user@ip-172-31-5-105 ~]$ /usr/local/nagios/libexec/check_disk -w 20% -c 10%
DISK OK - free space: / 7163 MiB (71.82% inode=99%); /dev 346 MiB (100.00% inode=100%); /dev/shm 453 MiB (100.00% inode=100%); /sys/firmware/efi/efivars 0 MiB (97.50% inode=); /run 172 MiB (95.19% inode=100%); /boot/efi 190 MiB (95.52% inode=); /run/credentials/getty@tty1.service 1 MiB (100.00% inode=100%); /run/credentials/serial-getty@tty50.service 1 MiB (100.00% inode=100%); /run/user/1000 90 MiB (99.99% inode=100%); /run/credentials/systemd-journald.service 1 MiB (100.00% inode=100%); /#2810MiB:7978;8975;0;9873 /dev=0MiB;276;311;0;346 /dev/shm=0MiB;362;407;0;453 /sys/firmware/efi/efivars=0MiB;0;0;0 /run=8MiB;144;162;0;181 /boot/efi=8MiB;159;179;0;199 /run/credentials/getty@tty1.service=0MiB;0;0;0;1 /run/credentials/serial-getty@tty50.service=0MiB;0;0;0;1 /run/user/1000=0MiB;72;81;0;90 /run/credentials/systemd-journald.service=0MiB;0;0;0;1
[ec2-user@ip-172-31-5-105 ~]$
```

CPU Load Monitoring

Server: servera

The check_load plugin was used to monitor system load:

```
/usr/local/nagios/libexec/check_load -w 5,4,3 -c 10,6,4
```

The command checked the system load average against the configured warning and critical thresholds.

Screenshot – CPU Load Monitoring

```
[ec2-user@ip-172-31-5-105 ~]$ /usr/local/nagios/libexec/check_load -w 5,4,3 -c 10,6,4
OK - load average: 0.01, 0.03, 0.00|load1=0.010;5.000;10.000;0; load5=0.030;4.000;6.000;0; load15=0.000;3.000;4.000;0;
[ec2-user@ip-172-31-5-105 ~]$ |
```

HTTP Monitoring

Server: servera

The HTTP service was checked:

```
/usr/local/nagios/libexec/check_http -H localhost
```

The initial request returned HTTP 403 because the root HTTP path was not directly accessible.

The Nagios web interface was then tested:

```
/usr/local/nagios/libexec/check_http -H localhost -u /nagios/
```

The unauthenticated request returned:

```
HTTP WARNING: HTTP/1.1 401 Unauthorized
```

This was expected because the Nagios web interface requires authentication.

An authenticated HTTP check was then performed, which returned **HTTP 200 OK**.

The password was not included in the documentation or screenshot.

Screenshot – HTTP Monitoring

```
[ec2-user@ip-172-31-5-105 ~]$ /usr/local/nagios/libexec/check_http -H localhost -u /nagios/ -a nagiosadmin:123
HTTP OK: HTTP/1.1 200 OK - 1138 bytes in 0.002 second response time |time=0.002059s;;;0.000000 size=11388;;0
[ec2-user@ip-172-31-5-105 ~]$
```

SSH Monitoring

Server: servera

The SSH service was checked using:

```
/usr/local/nagios/libexec/check_ssh localhost
```

The command successfully verified the SSH service.

Screenshot – SSH Monitoring

```
[ec2-user@ip-172-31-5-105 ~]$ /usr/local/nagios/libexec/check_ssh localhost
SSH OK - OpenSSH_9.9 (protocol 2.0) | time=0.009564s;;;0.000000;10.000000
[ec2-user@ip-172-31-5-105 ~]$
```

Process Monitoring

Server: servera

The number of running processes was checked:

```
/usr/local/nagios/libexec/check_procs -w 250 -c 400
```

The command returned a normal status because the number of processes was below the configured thresholds.

Screenshot – Process Monitoring

```
[ec2-user@ip-172-31-5-105 ~]$ /usr/local/nagios/libexec/check_procs -w 250 -c 400
PROCS OK: 135 processes | procs=135;250;400;0;
[ec2-user@ip-172-31-5-105 ~]$
```

Logged-in User Monitoring

Server: servera

The number of logged-in users was checked:

```
/usr/local/nagios/libexec/check_users -w 5 -c 10
```

The command returned a normal status.

Screenshot – User Monitoring

```
[ec2-user@ip-172-31-5-105 ~]$ /usr/local/nagios/libexec/check_users -w 5 -c 10
USERS OK - 3 users currently logged in |users=3;5;10;0
[ec2-user@ip-172-31-5-105 ~]$ |
```

Memory Monitoring

Server: servera

The free command was used to examine memory utilization:

```
free -h
```

The command displayed total, used, free, shared, buffer/cache and available memory.

Screenshot – Memory Monitoring

```
[ec2-user@ip-172-31-5-105 ~]$ free -h
              total        used        free      shared  buff/cache   available
Mem:           906Mi       481Mi       154Mi        13Mi        407Mi       425Mi
Swap:              0B           0B           0B
[ec2-user@ip-172-31-5-105 ~]$
```

CPU Monitoring

Server: servera

CPU utilization was checked using:

```
top -bn1 | grep "Cpu(s)"
```

The command displayed CPU usage including user, system and idle percentages.

Screenshot – CPU Monitoring

```
[ec2-user@ip-172-31-5-105 ~]$ top -bn1 | grep "Cpu(s)"
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
[ec2-user@ip-172-31-5-105 ~]$
```

Configure NRPE on the Nagios Client

NRPE was used to allow the Nagios server to execute monitoring plugins on the remote client.

Install NRPE Dependencies

Server: serverb

Required packages were installed:

```
sudo dnf install -y gcc make wget openssl-devel
```

Download NRPE

Server: serverb

NRPE 4.1.3 was downloaded:

```
wget https://github.com/NagiosEnterprises/nrpe/releases/download/nrpe-4.1.3/nrpe-4.1.3.tar.gz
```

The archive was extracted and the source directory was entered.

Compile and Install NRPE

Server: serverb

NRPE was configured:

```
./configure --enable-command-args
```

The source was compiled:

```
make all
```

A Nagios user and group were created:

```
sudo useradd nagios
sudo groupadd nagcmd
sudo usermod -aG nagcmd nagios
```

NRPE was installed:

```
sudo make install
sudo make install-config
sudo make install-init
```

The systemd configuration was reloaded:

```
sudo systemctl daemon-reload
```

Start and Verify NRPE

Server: serverb

NRPE was started:

```
sudo systemctl start nrpe
```

NRPE was enabled:

```
sudo systemctl enable nrpe
```

The service status was checked:

```
sudo systemctl status nrpe --no-pager
```

The service was successfully running.

Screenshot – NRPE Service

```
[ec2-user@ip-172-31-4-76 ~]$ sudo systemctl status nrpe --no-pager
● nrpe.service - Nagios Remote Plugin Executor
   Loaded: loaded (/usr/lib/systemd/system/nrpe.service; disabled; preset: disable
d)
   Active: active (running) since Tue 2026-09-22 07:27:28 UTC; 2h 30min ago
 Invocation: fa19cb130a284c4fa3113cf46a888754
    Docs: http://www.nagios.org/documentation
   Main PID: 18649 (nrpe)
     Tasks: 1 (limit: 4156)
    Memory: 1.8M (peak: 4.1M)
       CPU: 497ms
    CGroup: /system.slice/nrpe.service
            └─18649 /usr/local/nagios/bin/nrpe -c /usr/local/nagios/etc/nrpe.cfg -f
```

Verify NRPE Port

Server: serverb

NRPE listens on TCP port 5666.

The port was verified using:

```
sudo ss -lntp | grep 5666
```

The output showed NRPE listening on port 5666.

15. Configure NRPE Access

Server: serverb

The NRPE configuration file was:

```
/usr/local/nagios/etc/nrpe.cfg
```

The allowed host was configured as:

```
allowed_hosts=172.31.5.105,::1
```

This allowed the Nagios server (servera) to communicate with NRPE on serverb.

NRPE was restarted after the configuration change.

Test NRPE Connectivity

Test Port Connectivity

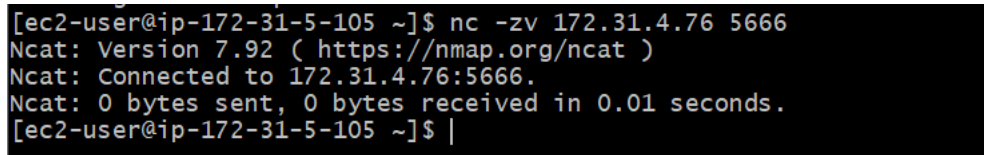
Server: servera

The connection to the NRPE port on serverb was tested:

```
nc -zv 172.31.4.76 5666
```

The connection was successful.

Screenshot – NRPE Port Connectivity



```
[ec2-user@ip-172-31-5-105 ~]$ nc -zv 172.31.4.76 5666
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Connected to 172.31.4.76:5666.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.
[ec2-user@ip-172-31-5-105 ~]$ |
```

Test NRPE Communication

Server: servera

The check_nrpe plugin was used:

```
sudo /usr/local/nagios/libexec/check_nrpe -H 172.31.4.76
```

The result was:

```
NRPE v4.1.3
```

This confirmed successful communication between the Nagios server and client.

Install Nagios Plugins on Client

Server: serverb

Nagios Plugins 2.4.12 were installed on the client so that NRPE could execute monitoring commands locally.

The plugins were configured and installed:

```
./configure --with-nagios-user=nagios --with-nagios-group=nagios  
make  
sudo make install
```

The `check_load`, `check_disk`, `check_users` and other plugins became available on `serverb`.

Configure Remote Monitoring Commands

Server: `serverb`

The following NRPE commands were configured.

Load

```
command[check_load]=/usr/local/nagios/libexec/check_load -r -  
w .15,.10,.05 -c .30,.25,.20
```

Disk

```
command[check_disk]=/usr/local/nagios/libexec/check_disk -w 20% -c  
10% -p /
```

Users

```
command[check_users]=/usr/local/nagios/libexec/check_users -w 5 -c  
10
```

These commands allowed the Nagios server to remotely check load, disk usage and logged-in users on `serverb`.

Test Remote Monitoring

Remote Load Check

Server: servera

Target: serverb

The load on serverb was checked remotely:

```
sudo /usr/local/nagios/libexec/check_nrpe -H 172.31.4.76 -c  
check_load
```

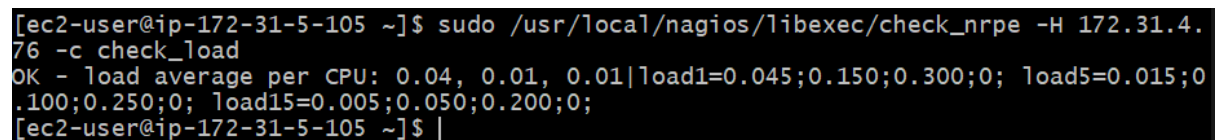
Under normal conditions, the command returned an **OK** status with the load average of serverb.

Example:

OK - load average per CPU: 0.05, 0.05, 0.02

The output also displayed the warning and critical thresholds.

Screenshot – Remote Client Load Check

A terminal window showing the execution of the check_nrpe command to check the load on serverb. The output shows an OK status with load averages per CPU and various thresholds.

```
[ec2-user@ip-172-31-5-105 ~]$ sudo /usr/local/nagios/libexec/check_nrpe -H 172.31.4.  
76 -c check_load  
OK - load average per CPU: 0.04, 0.01, 0.01|load1=0.045;0.150;0.300;0; load5=0.015;0  
.100;0.250;0; load15=0.005;0.050;0.200;0;  
[ec2-user@ip-172-31-5-105 ~]$ |
```

Remote Disk Check

Server: servera

Target: serverb

The disk on serverb was checked remotely:

```
sudo /usr/local/nagios/libexec/check_nrpe -H 172.31.4.76 -c  
check_disk
```

The result returned an **OK** status.

Screenshot – Remote Client Disk Check

```
[ec2-user@ip-172-31-5-105 ~]$ sudo /usr/local/nagios/libexec/check_nrpe -H 172.31.4.76 -c check_disk
DISK OK - free space: / 7649 MiB (76.69% inode=99%);| /=2324MiB;7978;8975;0;9973
[ec2-user@ip-172-31-5-105 ~]$
```

Remote User Check

Server: servera

Target: serverb

The number of logged-in users on serverb was checked:

```
sudo /usr/local/nagios/libexec/check_nrpe -H 172.31.4.76 -c
check_users
```

The result returned an **OK** status.

Screenshot – Remote Client User Check

```
[ec2-user@ip-172-31-5-105 ~]$ sudo /usr/local/nagios/libexec/check_nrpe -H 172.31.4.76 -c check_users
USERS OK - 2 users currently logged in |users=2;5;10;0
[ec2-user@ip-172-31-5-105 ~]$
```

Add serverb as a Nagios Host

Server: servera

A directory was created for additional host configurations:

```
sudo mkdir -p /usr/local/nagios/etc/servers
```

The following configuration directory was added to `nagios.cfg`:

```
cfg_dir=/usr/local/nagios/etc/servers
```

A configuration file was created:

```
/usr/local/nagios/etc/servers/serverb.cfg
```

The serverb host was configured:

```
define host {
    use                linux-server
    host_name          serverb
    alias              Nagios Client - serverb
    address            172.31.4.76
    max_check_attempts 5
    check_period       24x7
    notification_interval 30
    notification_period 24x7
}
```

Configure Nagios NRPE Command

Server: servera

The check_nrpe command was added to the Nagios command configuration:

```
define command {
    command_name    check_nrpe
    command_line    $USER1$/check_nrpe -H $HOSTADDRESS$ -c $ARG1$
}
```

This allowed Nagios to execute NRPE checks on serverb.

Add Load Service

Server: servera

The Load service was added to serverb:

```
define service {
    use                generic-service
    host_name          serverb
    service_description Load
    check_command       check_nrpe!check_load
}
```

Add Disk Service

Server: servera

The Disk service was added:

```
define service {
    use                generic-service
    host_name          serverb
    service_description Disk
    check_command       check_nrpe!check_disk
}
```

Add Users Service

Server: servera

The Users service was added:

```
define service {
    use                generic-service
    host_name          serverb
    service_description Users
    check_command       check_nrpe!check_users
}
```

Validate Nagios Configuration

Server: servera

After adding the host and services, the Nagios configuration was validated:

```
sudo /usr/local/nagios/bin/nagios -v
/usr/local/nagios/etc/nagios.cfg
```

The configuration was validated successfully.

Nagios was restarted:

```
sudo systemctl restart nagios
```

Verify serverb Host Status

The Nagios web interface was opened and the serverb host was checked.

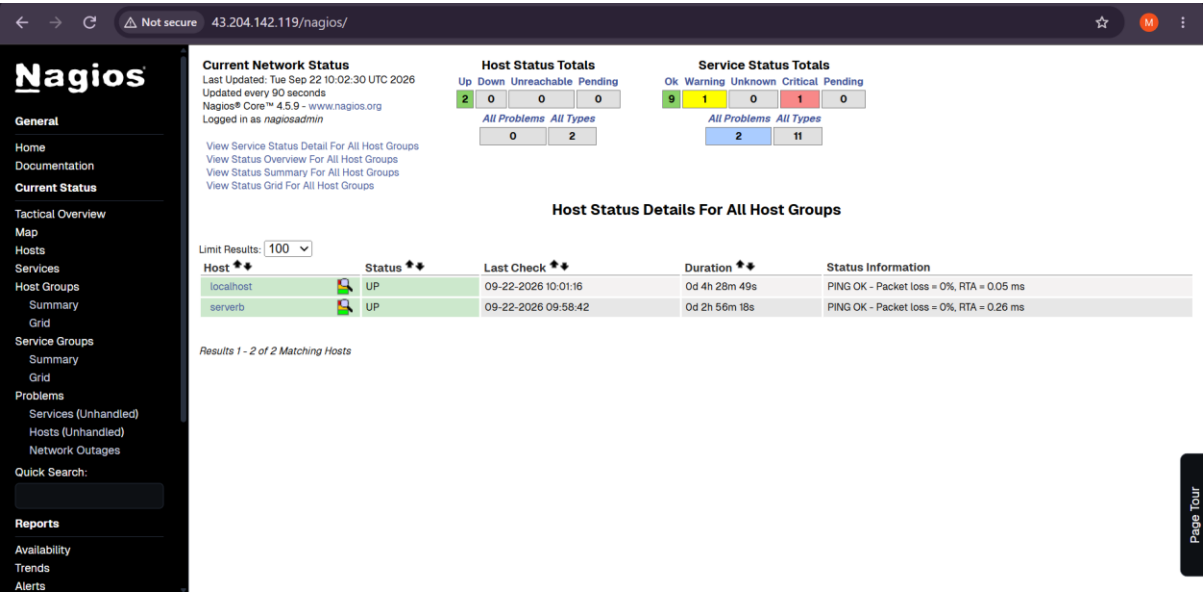
Initially, the host check showed a problem because ICMP traffic was not allowed.

The required **All ICMP - IPv4** security group rule was then configured.

After the security group modification, the host status changed to:

serverb - UP

Screenshot – serverb Host UP



Verify serverb Services

The Nagios web interface was checked for the configured services.

The following services were monitored:

- Load
- Disk

- Users

Under normal conditions, the services showed:

Load - OK

Disk - OK

Users - OK

Screenshot – serverb Services OK

The screenshot shows the Nagios web interface at 43.204.142.119/nagios/. The interface includes a sidebar with navigation links (General, Home, Documentation, Current Status, Tactical Overview, Map, Hosts, Services, Host Groups, Summary, Grid, Service Groups, Summary, Grid, Problems, Services (Unhandled), Hosts (Unhandled), Network Outages, Quick Search, Reports, Availability, Trends, Alerts) and a main content area. The main content area displays 'Current Network Status', 'Host Status Totals', 'Service Status Totals', and 'Service Status Details For All Hosts'. The 'Service Status Details For All Hosts' table shows the following data:

Host	Service	Status	Last Check	Duration	Attempt	Status Information
localhost	Current Load	OK	09-22-2026 10:02:16	0d 3h 41m 14s	1/4	OK - load average: 0.00, 0.00, 0.00
	Current Users	OK	09-22-2026 09:59:54	0d 4h 28m 36s	1/4	USERS OK - 3 users currently logged in
	HTTP	WARNING	09-22-2026 09:58:31	0d 4h 24m 59s	4/4	HTTP WARNING: HTTP/1.1 403 Forbidden - 6199 bytes in 0.001 second response time
	PING	OK	09-22-2026 10:01:09	0d 4h 27m 21s	1/4	PING OK - Packet loss = 0%, RTA = 0.04 ms
	Root Partition	OK	09-22-2026 10:01:46	0d 4h 26m 44s	1/4	DISK OK - free space: / 7163 MiB (71.81% inode=99%):
	SSH	OK	09-22-2026 10:02:24	0d 4h 26m 6s	1/4	SSH OK - OpenSSH_9.9 (protocol 2.0)
serverb	Swap Usage	CRITICAL	09-22-2026 10:01:01	0d 4h 22m 29s	4/4	SWAP CRITICAL - 0% free (0 MB out of 0 MB) - Swap is either disabled, not present, or of zero size.
	Total Processes	OK	09-22-2026 09:58:39	0d 4h 24m 51s	1/4	PROCS OK: 40 processes with STATE = RSZDT
	Disk	OK	09-22-2026 10:00:59	0d 2h 52m 31s	1/3	DISK OK - free space: / 7649 MiB (76.69% inode=99%):
	Load	OK	09-22-2026 09:58:42	0d 1h 44m 48s	1/3	OK - load average per CPU: 0.00, 0.00, 0.00
	Users	OK	09-22-2026 10:00:02	0d 2h 33m 28s	1/3	USERS OK - 2 users currently logged in

Results 1 - 11 of 11 Matching Services

Client CPU Load Monitoring

The CPU/system load of the remote Nagios client, serverb, was monitored from the Nagios server using NRPE.

Check Client Load Directly

Server: servera

Target: serverb

The check_nrpe plugin was used to remotely execute the check_load plugin on serverb:

```
sudo /usr/local/nagios/libexec/check_nrpe -H 172.31.4.76 -c check_load
```

Under normal conditions, the command returned an **OK** status.

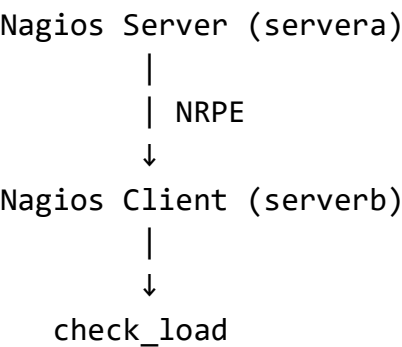
The output displayed the load average of the remote client along with its warning and critical thresholds.

Monitor Client Load Through Nagios Web Interface

Location: Nagios Web Interface

The Load service configured for serverb was checked from the Nagios dashboard.

The monitoring process was:



Under normal conditions, the Nagios web interface displayed:

serverb | Load | OK

Screenshot – Client Load in Nagios

The screenshot shows the Nagios web interface at 43.204.142.119/nagios/. The top status bar indicates the system is 'Not secure'. The main content area displays the 'Service Status Details For All Hosts' table, which lists the status of various services on the 'serverb' host.

Host	Service	Status	Last Check	Duration	Attempt	Status Information
localhost	Current Load	OK	09-22-2026 10:02:16	0d 3h 41m 14s	1/4	OK - load average: 0.00, 0.00, 0.00
	Current Users	OK	09-22-2026 09:59:54	0d 4h 28m 36s	1/4	USERS OK - 3 users currently logged in
	HTTP	WARNING	09-22-2026 09:58:31	0d 4h 24m 59s	4/4	HTTP WARNING: HTTP/1.1 403 Forbidden - 6199 bytes in 0.001 second response time
	PING	OK	09-22-2026 10:01:09	0d 4h 27m 21s	1/4	PING OK - Packet loss = 0%, RTA = 0.04 ms
	Root Partition	OK	09-22-2026 10:01:46	0d 4h 26m 44s	1/4	DISK OK - free space: / 7163 MiB (71.81% inode=99%):
	SSH	OK	09-22-2026 10:02:24	0d 4h 26m 6s	1/4	SSH OK - OpenSSH_9.9 (protocol 2.0)
	Swap Usage	CRITICAL	09-22-2026 10:01:01	0d 4h 22m 29s	4/4	SWAP CRITICAL - 0% free (0 MB out of 0 MB) - Swap is either disabled, not present, or of zero size.
serverb	Total Processes	OK	09-22-2026 09:58:39	0d 4h 24m 51s	1/4	PROCS OK: 40 processes with STATE = RSZDT
	Disk	OK	09-22-2026 10:00:59	0d 2h 52m 31s	1/3	DISK OK - free space: / 7649 MiB (76.69% inode=99%):
	Load	OK	09-22-2026 09:58:42	0d 1h 44m 48s	1/3	OK - load average per CPU: 0.00, 0.00, 0.00
	Users	OK	09-22-2026 10:00:02	0d 2h 33m 28s	1/3	USERS OK - 2 users currently logged in

Results 1 - 11 of 11 Matching Services

Client CPU Load Stress Test and Recovery

This test was performed to verify that Nagios could detect a real increase in system load on the remote client and automatically recover when the load was removed.

The test demonstrated:

OK → CRITICAL → OK

29.1 Generate CPU Load on Client

Server: serverb

Four CPU-intensive processes were started:

```
for i in {1..4}; do yes > /dev/null & done
```

The yes command continuously generated CPU activity.

29.2 Verify CPU Usage on Client

Server: serverb

The running processes were checked:

```
ps -eo pid,comm,%cpu --sort=-%cpu | head
```

The output showed multiple yes processes consuming CPU resources.

29.3 Verify CRITICAL Status in Nagios

Location: Nagios Web Interface

After generating CPU load on serverb, Nagios detected the increased system load through NRPE.

The serverb Load service changed from **OK** to **CRITICAL**.

The Nagios interface showed:

serverb | Load | CRITICAL

Screenshot – Client Load CRITICAL

The screenshot shows the Nagios web interface at 43.204.142.119/nagios/. The interface includes a sidebar with navigation links and a main content area displaying the current network status and service status details.

Current Network Status
Last Updated: Tue Sep 22 07:34:54 UTC 2026
Updated every 90 seconds
Nagios® Core™ 4.5.9 - www.nagios.org
Logged in as nagiosadmin

Host Status Totals

Up	Down	Unreachable	Pending
2	0	0	0

Service Status Totals

Ok	Warning	Unknown	Critical	Pending
8	1	0	2	0

Service Status Details For All Hosts

Limit Results: 100

Host	Service	Status	Last Check	Duration	Attempt	Status Information
localhost	Current Load	OK	09-22-2026 07:32:16	0d 1h 12m 38s	1/4	OK - load average: 0.00, 0.00, 0.00
	Current Users	OK	09-22-2026 07:29:54	0d 2h 0m 0s	1/4	USERS OK - 9 users currently logged in
	HTTP	WARNING	09-22-2026 07:33:31	0d 1h 56m 23s	4/4	HTTP WARNING: HTTP/1.1 403 Forbidden - 6199 bytes in 0.001 second response time
	PING	OK	09-22-2026 07:31:09	0d 1h 58m 45s	1/4	PING OK - Packet loss = 0%, RTA = 0.05 ms
	Root Partition	OK	09-22-2026 07:31:46	0d 1h 58m 8s	1/4	DISK OK - free space: / 7162 MiB (71.81% inode=99%):
	SSH	OK	09-22-2026 07:32:24	0d 1h 57m 30s	1/4	SSH OK - OpenSSH_9.9 (protocol 2.0)
serverb	Swap Usage	CRITICAL	09-22-2026 07:31:01	0d 1h 53m 53s	4/4	SWAP CRITICAL - 0% free (0 MB out of 0 MB) - Swap is either disabled, not present, or of zero size.
	Total Processes	OK	09-22-2026 07:33:39	0d 1h 56m 15s	1/4	PROCS OK: 47 processes with STATE = RSZDT
	Disk	OK	09-22-2026 07:30:59	0d 0h 23m 55s	1/3	DISK OK - free space: / 7649 MiB (76.69% inode=99%):
	Load	CRITICAL	09-22-2026 07:34:42	0d 0h 0m 12s	1/3	CRITICAL - load average per CPU: 1.99, 1.25, 0.54

Results 1 - 11 of 11 Matching Services

This demonstrated that the Nagios server successfully detected the high load condition on the remote client.

29.4 Stop the CPU Load

Server: serverb

The CPU-intensive processes were stopped:

```
kill yes
```

29.5 Verify Load Recovery

Server: serverb

The system load was checked:

```
uptime
```

The load gradually decreased after the CPU-intensive processes were stopped.

29.6 Confirm Nagios Recovery

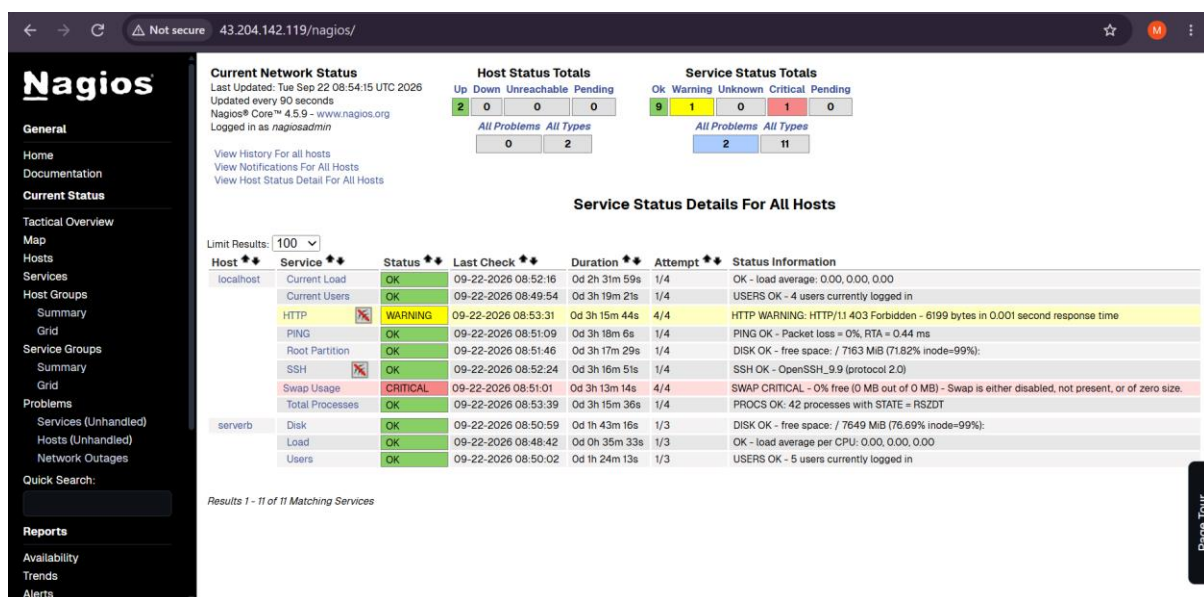
Location: Nagios Web Interface

After the system load returned below the configured thresholds, Nagios automatically changed the Load service from **CRITICAL** back to **OK**.

The final Nagios status showed:

serverb | Load | OK

Screenshot – Client Load OK After Recovery



This confirmed that Nagios successfully detected both the abnormal condition and the recovery.

30. Important Nagios Monitoring Commands Studied

The following commands were studied and tested during the practical.

Command	Purpose	Server
check_disk	Checks disk usage	servera/serverb
check_load	Checks system load	servera/serverb

check_http	Checks HTTP service	servera
check_ssh	Checks SSH service	servera
check_procs	Checks running processes	servera
check_users	Checks logged-in users	servera/serverb
free -h	Displays memory usage	servera
top	Displays CPU/process information	servera
uptime	Displays system uptime and load average	serverb
check_nrpe	Communicates with remote NRPE client	servera
nc -zv	Tests TCP connectivity	servera

31. Important Concepts Learned

Nagios Core

Nagios Core is the main monitoring engine that checks hosts and services and displays their status.

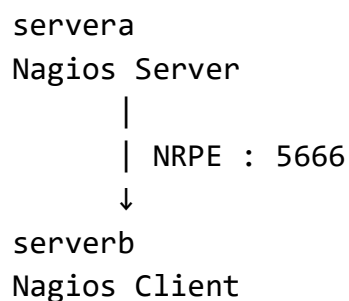
Nagios Plugins

Nagios Plugins perform individual monitoring checks such as disk, load, HTTP, SSH and processes.

NRPE

NRPE allows the Nagios server to execute monitoring plugins on a remote Linux server.

In this practical:



Host Monitoring

Host monitoring checks whether the remote server itself is available.

In this practical, serverb was monitored as a Nagios host.

Service Monitoring

Service monitoring checks individual resources or services running on the host.

The following services were configured on serverb:

Load
Disk
Users

Monitoring States

Nagios uses states such as:

OK
WARNING
CRITICAL
UNKNOWN

The client CPU load test demonstrated:

Normal Load
↓
OK
↓
CPU Load Generated
↓
CRITICAL
↓
CPU Load Stopped
↓
Recovery
↓
OK

32. Conclusion

The Nagios server monitoring practical was successfully completed using two RHEL 10.2 AWS EC2 instances.

Nagios Core and Nagios Plugins were installed and configured on `servera`, while NRPE and Nagios Plugins were configured on `serverb` for remote monitoring.

The Nagios server successfully monitored the remote `serverb` host and its **Load, Disk and Users** services. Communication between the two servers was verified through NRPE on TCP port 5666.

Various monitoring commands were also studied and tested for disk usage, CPU load, memory, processes, logged-in users, HTTP and SSH services.

The CPU/system load of the remote client was monitored directly through NRPE and through the Nagios web interface. A CPU load was then intentionally generated on `serverb`. Nagios detected the increased load and changed the Load service status from **OK** to **CRITICAL**. After the CPU-intensive processes were stopped and the system load recovered, Nagios automatically changed the status back to **OK**.

This confirmed the successful installation, configuration and practical operation of Nagios for server monitoring, remote service monitoring, fault detection and recovery.

Zabbix Server and Agent Monitoring

Description

The objective of this practical was to install and configure **Zabbix Server** for centralized monitoring and configure **Zabbix Agent 2** on two RHEL EC2 instances. Both the Zabbix server and client server were added to the Zabbix monitoring system. CPU/load monitoring was tested by generating system load and verifying that Zabbix detected the changes and displayed the collected data.

Infrastructure Used

Server	Role	Private IP
<code>servera</code>	Zabbix Server + Zabbix Agent 2	172.31.5.105
<code>serverb</code>	Zabbix Agent 2	172.31.4.76

Both instances were RHEL 10.2 x86_64 EC2 instances.

Task 1 – Install Zabbix Repository and Server Packages

The Zabbix 7.4 repository was added to servera.

On servera

```
sudo rpm -Uvh https://repo.zabbix.com/zabbix/7.4/release/rhel/10/noarch/zabbix-release-latest-7.4.el10.noarch.rpm
sudo dnf clean all
sudo dnf repolist | grep -i zabbix
```

The required Zabbix Server, web interface, Apache configuration, SQL scripts and SELinux policy packages were installed.

```
sudo dnf install -y zabbix-server-mysql zabbix-web-mysql zabbix-apache-conf zabbix-sql-scripts zabbix-selinux-policy
```

Task 2 – Install and Configure MariaDB

MariaDB was installed on servera to provide the database backend for Zabbix.

```
sudo dnf install -y mariadb-server
sudo systemctl enable --now mariadb
sudo systemctl status mariadb --no-pager
```

A database named zabbix and a database user named zabbix were created.

```
CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;
CREATE USER 'zabbix'@'localhost' IDENTIFIED BY '123';
GRANT ALL PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost';
FLUSH PRIVILEGES;
```

The Zabbix database schema was imported using:

```
zcat /usr/share/zabbix/sql-scripts/mysql/server.sql.gz | mysql -uzabbix -p zabbix
```

During the import, the message:

ERROR 1050 (42S01) at line 1: Table 'role' already exists

was displayed because the Zabbix database schema had already been imported.

The database was verified using:

```
sudo mysql -uzabbix -p -e "USE zabbix; SHOW TABLES;" | head -15
```

The database contained **207 tables**.

```
mysql -uzabbix -p -e "USE zabbix; SELECT COUNT(*) AS tables_count  
FROM information_schema.tables WHERE table_schema='zabbix';"
```

Task 3 – Configure Zabbix Server

The Zabbix server configuration file was edited:

```
sudo vi /etc/zabbix/zabbix_server.conf
```

The database configuration was set as:

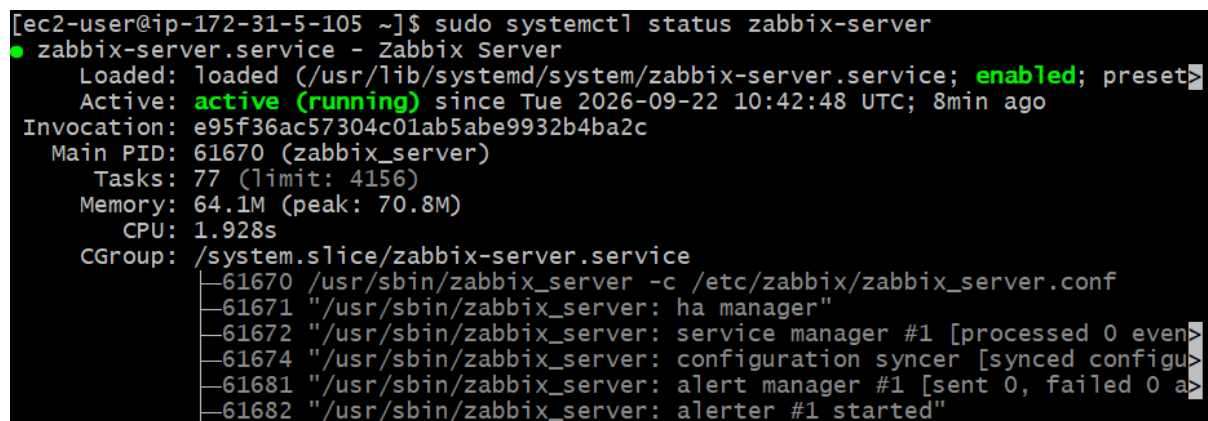
```
DBName=zabbix  
DBUser=zabbix  
DBPassword=123
```

The Zabbix server service was then enabled and started:

```
sudo systemctl enable --now zabbix-server  
sudo systemctl status zabbix-server --no-pager
```

The Zabbix server was confirmed to be running successfully.

Screenshot Output



```
[ec2-user@ip-172-31-5-105 ~]$ sudo systemctl status zabbix-server
● zabbix-server.service - Zabbix Server
   Loaded: loaded (/usr/lib/systemd/system/zabbix-server.service; enabled; preset>
   Active: active (running) since Tue 2026-09-22 10:42:48 UTC; 8min ago
   Invocation: e95f36ac57304c01ab5abe9932b4ba2c
   Main PID: 61670 (zabbix_server)
     Tasks: 77 (limit: 4156)
    Memory: 64.1M (peak: 70.8M)
       CPU: 1.928s
    CGroup: /system.slice/zabbix-server.service
            └─61670 /usr/sbin/zabbix_server -c /etc/zabbix/zabbix_server.conf
              └─61671 "/usr/sbin/zabbix_server: ha manager"
                └─61672 "/usr/sbin/zabbix_server: service manager #1 [processed 0 even>
                  └─61674 "/usr/sbin/zabbix_server: configuration syncer [syncd configu>
                    └─61681 "/usr/sbin/zabbix_server: alert manager #1 [sent 0, failed 0 a>
                      └─61682 "/usr/sbin/zabbix_server: alerter #1 started"
```

Task 4 – Configure Apache and PHP

Apache HTTP Server was enabled and started:

```
sudo systemctl enable --now httpd
sudo systemctl status httpd --no-pager
```

The PHP timezone was configured in:

`/etc/php.ini`

The following timezone was set:

```
date.timezone = Asia/Kolkata
```

Apache was restarted after the configuration change.

```
sudo systemctl restart httpd
```

The Zabbix web interface was then accessed using:

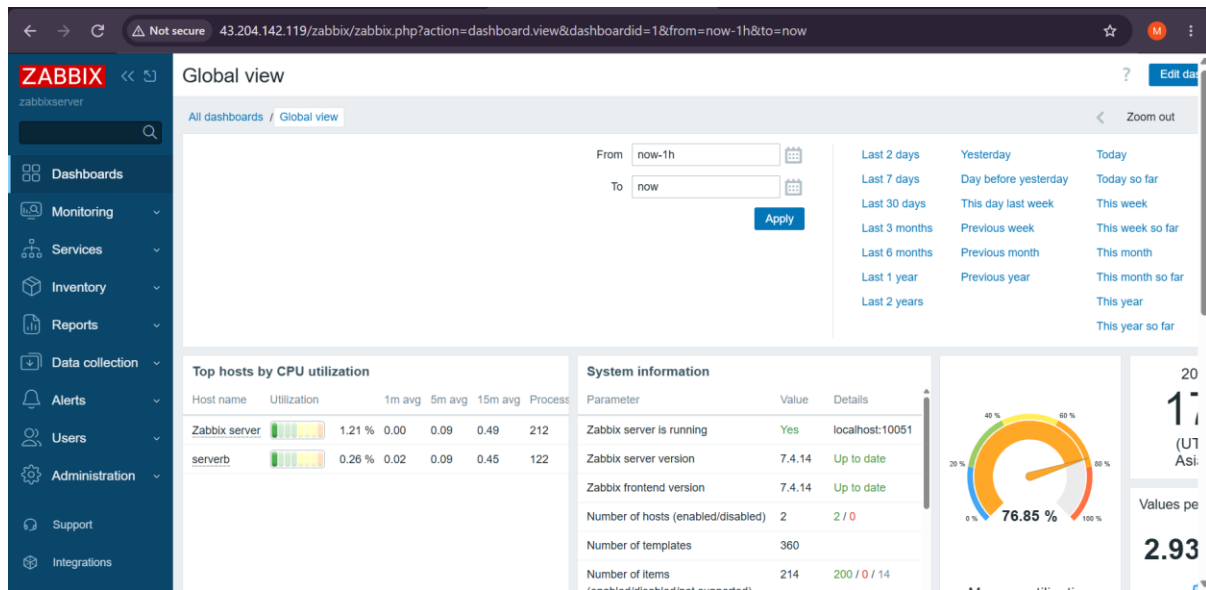
<http://43.204.142.119/zabbix>

The Zabbix web setup wizard was completed using:

- Database type: MySQL
- Database host: localhost
- Database port: 3306
- Database name: zabbix
- Database user: zabbix
- Database password: configured database password

The Zabbix dashboard was successfully opened.

Screenshot Output



Task 5 – Install Zabbix Agent 2 on serverb

The Zabbix repository was installed on serverb.

On serverb

```
sudo rpm -Uvh https://repo.zabbix.com/zabbix/7.4/release/rhel/10/noarch/zabbix-release-latest-7.4.el10.noarch.rpm
```

```
sudo dnf clean all
```

Zabbix Agent 2 was installed:

```
sudo dnf install -y zabbix-agent2
```

The installed version was:

```
zabbix-agent2-7.4.14-release1.el10.x86_64
```

Task 6 – Configure Zabbix Agent 2 on serverb

The agent configuration file was edited:

```
sudo vi /etc/zabbix/zabbix_agent2.conf
```

The following parameters were configured:

Server=172.31.5.105

ServerActive=172.31.5.105

Hostname=serverb

The agent was enabled and started:

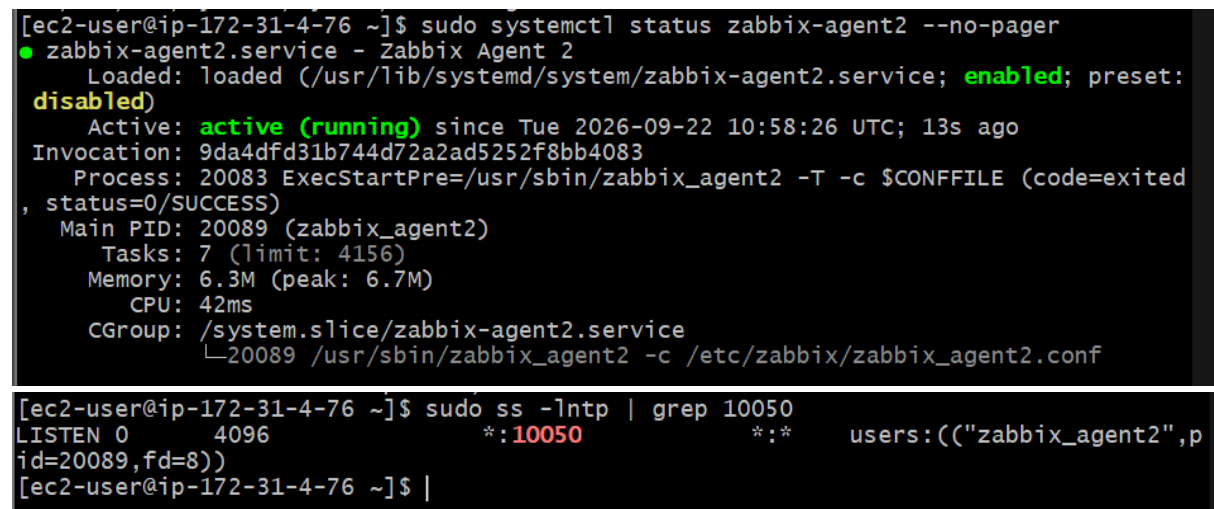
```
sudo systemctl enable --now zabbix-agent2
```

```
sudo systemctl status zabbix-agent2 --no-pager
```

The agent was confirmed to be listening on port 10050.

```
sudo ss -lntp | grep 10050
```

Screenshot Output



```
[ec2-user@ip-172-31-4-76 ~]$ sudo systemctl status zabbix-agent2 --no-pager
● zabbix-agent2.service - Zabbix Agent 2
   Loaded: loaded (/usr/lib/systemd/system/zabbix-agent2.service; enabled; preset:
disabled)
   Active: active (running) since Tue 2026-09-22 10:58:26 UTC; 13s ago
 Invocation: 9da4dfd31b744d72a2ad5252f8bb4083
   Process: 20083 ExecStartPre=/usr/sbin/zabbix_agent2 -T -c $CONFFILE (code=exited
, status=0/SUCCESS)
   Main PID: 20089 (zabbix_agent2)
    Tasks: 7 (limit: 4156)
   Memory: 6.3M (peak: 6.7M)
      CPU: 42ms
   CGroup: /system.slice/zabbix-agent2.service
           └─20089 /usr/sbin/zabbix_agent2 -c /etc/zabbix/zabbix_agent2.conf

[ec2-user@ip-172-31-4-76 ~]$ sudo ss -lntp | grep 10050
LISTEN 0          4096             *:10050          :*               users:((("zabbix_agent2",p
id=20089,fd=8))
[ec2-user@ip-172-31-4-76 ~]$ |
```

Task 7 – Test Communication Between servera and serverb

The Zabbix agent port was tested from servera.

On servera

```
nc -zv 172.31.4.76 10050
```

The connection was successful:

Ncat: Connected to 172.31.4.76:10050.

The zabbix-get package was installed:

```
sudo dnf install -y zabbix-get
```

The Zabbix agent was tested using:

```
zabbix_get -s 172.31.4.76 -k agent.ping
```

The result was:

1

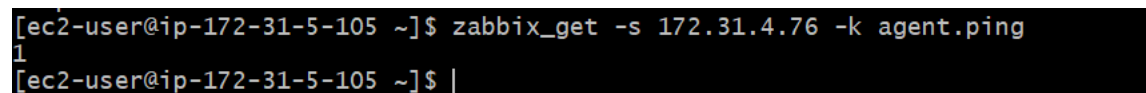
This confirmed that servera could successfully communicate with the Zabbix Agent on serverb.

The CPU load was also queried:

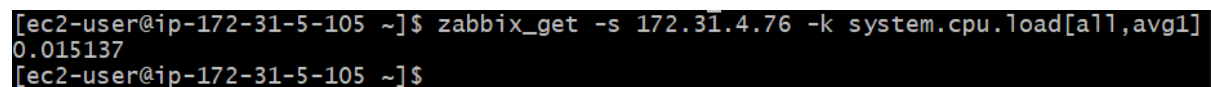
```
zabbix_get -s 172.31.4.76 -k system.cpu.load[all,avg1]
```

A CPU load value was returned successfully.

Screenshot Output



```
[ec2-user@ip-172-31-5-105 ~]$ zabbix_get -s 172.31.4.76 -k agent.ping
1
[ec2-user@ip-172-31-5-105 ~]$ |
```



```
[ec2-user@ip-172-31-5-105 ~]$ zabbix_get -s 172.31.4.76 -k system.cpu.load[all,avg1]
0.015137
[ec2-user@ip-172-31-5-105 ~]$
```

Task 8 – Add serverb to Zabbix

The serverb host was added through the Zabbix web interface.

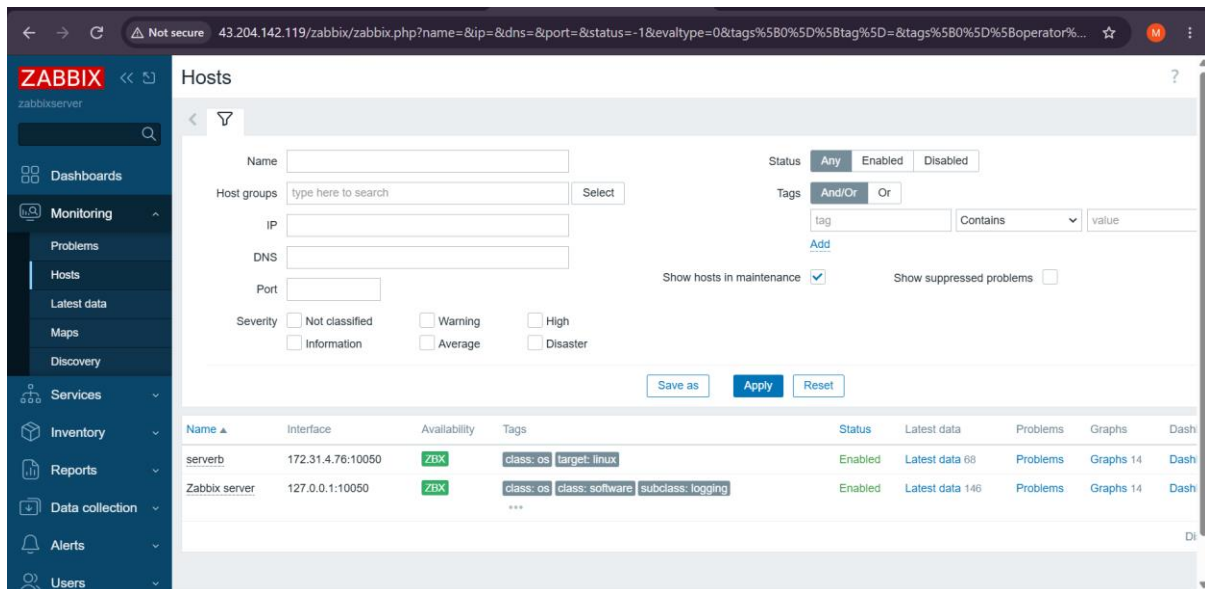
The following configuration was used:

- Host name: serverb
- Host group: Linux servers
- Interface type: Zabbix Agent
- IP address: 172.31.4.76
- Port: 10050
- Template: **Linux by Zabbix agent**

After saving the configuration, the Zabbix availability indicator for serverb became **green**.

This confirmed that the Zabbix server was successfully communicating with the agent.

Screenshot Output



Task 9 – Install Zabbix Agent 2 on the Zabbix Server

The Zabbix server itself was also monitored by installing Zabbix Agent 2 on servera.

On servera

```
sudo dnf install -y zabbix-agent2
```

The agent configuration file was edited:

```
sudo vi /etc/zabbix/zabbix_agent2.conf
```

The following configuration was used:

```
Server=127.0.0.1
ServerActive=127.0.0.1
Hostname=Zabbix server
```

The agent was started:

```
sudo systemctl enable --now zabbix-agent2  
sudo systemctl status zabbix-agent2 --no-pager
```

Port 10050 was verified:

```
sudo ss -lntp | grep 10050
```

The local agent was tested:

```
zabbix_get -s 127.0.0.1 -k agent.ping
```

The result was:

1

The CPU load was also tested:

```
zabbix_get -s 127.0.0.1 -k system.cpu.load[all,avg1]
```

A valid CPU load value was returned.

Task 10 – Verify Both Hosts in Zabbix

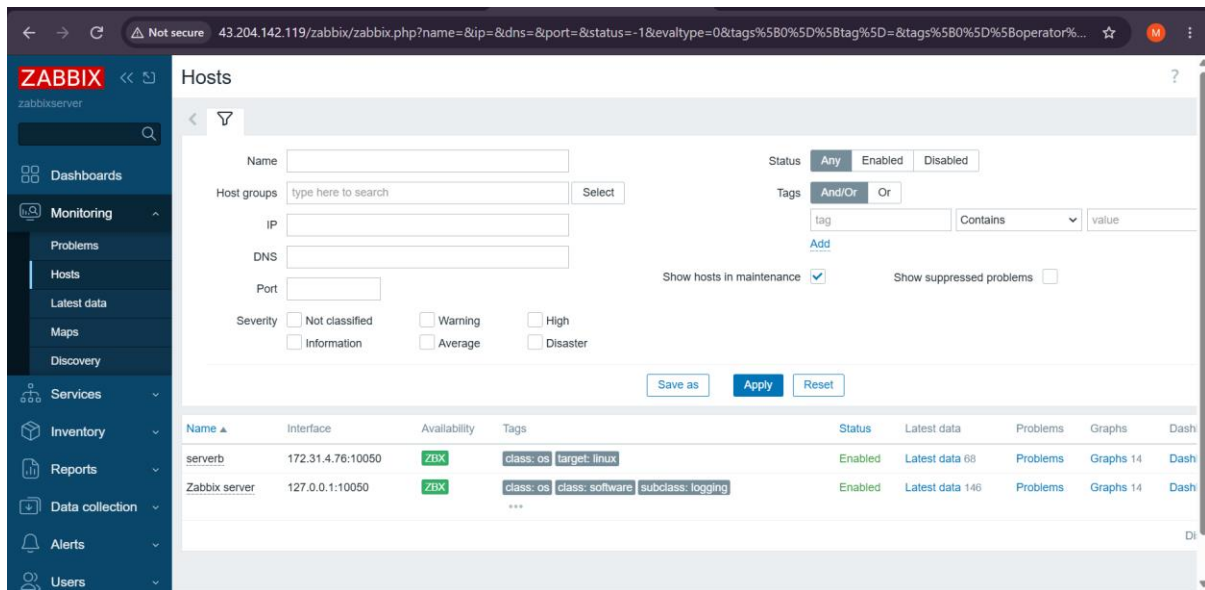
The Zabbix **Monitoring → Hosts** page was checked.

The following hosts were available:

```
Zabbix server  
serverb
```

Both hosts showed **green ZBX availability**, confirming that Zabbix was successfully collecting monitoring information from both systems.

Screenshot Output



Task 11 – Generate CPU Load on Both Servers

A CPU stress test was performed to verify that Zabbix could detect changes in system load.

On servera

The normal load was first checked:

```
uptime
```

CPU-consuming processes were then started:

```
for i in {1..4}; do yes > /dev/null & done
```

The running processes were verified:

```
ps -eo pid,comm,%cpu --sort=-%cpu | head
```

Multiple yes processes were shown consuming CPU resources.

On serverb

The normal load was checked:

```
uptime
```

CPU-consuming processes were started:

```
for i in {1..4}; do yes > /dev/null & done
```

The processes were verified:

```
ps -eo pid,comm,%cpu --sort=-%cpu | head
```

Multiple yes processes were shown consuming CPU resources.

Task 12 – Verify Increased Load in Zabbix

After generating CPU load, the **Monitoring → Latest data** section was opened in Zabbix.

For the **Zabbix server**, the recorded load values reached approximately:

- Load average (1m): **3.9268**
- Load average (5m): **2.1167**
- Load average (15m): **0.9092**

For **serverb**, the recorded load values reached approximately:

- Load average (1m): **3.9297**
- Load average (5m): **2.2456**
- Load average (15m): **0.9209**

The increased 1-minute load demonstrated that Zabbix was successfully collecting the CPU/load information while the servers were under stress.

Screenshot Output

The screenshot shows the Zabbix web interface. The left sidebar contains navigation links: Dashboards, Monitoring (selected), Problems, Hosts, Latest data, Maps, Discovery, Services, Inventory, Reports, Data collection, Alerts, and Users. The main content area displays monitoring data for the host 'Zabbix server'. The 'Hosts' section shows 'Zabbix server' with 3 items. The 'Tags' section shows 'component: cpu' with 3 values. The 'TAG VALUES' section shows 'component: cpu' with 3 values. The 'DATA' section shows 'With data' and 'Without data' tabs. The 'With data' tab is active, displaying a table with columns: Host, Name, Last check, Last value, Change, and Tags. The table contains three rows of data for the 'Zabbix server' host, showing load average (1m avg), load average (5m avg), and load average (15m avg) for the 'component: cpu' tag.

Host	Name	Last check	Last value	Change	Tags
Zabbix server	Load average (1m avg)	40s	3.9268	+0.2432	component: cpu
Zabbix server	Load average (5m avg)	32s	2.1167	+0.4404	component: cpu
Zabbix server	Load average (15m avg)	30s	0.9092	+0.2148	component: cpu

The screenshot shows the Zabbix web interface. The left sidebar contains navigation links: Dashboards, Monitoring (selected), Problems, Hosts, Latest data, Maps, Discovery, Services, Inventory, Reports, Data collection, Alerts, and Users. The main content area displays monitoring data for the host 'serverb'. The 'Hosts' section shows 'serverb' with 3 items. The 'Tags' section shows 'component: cpu' with 3 values. The 'TAG VALUES' section shows 'component: cpu' with 3 values. The 'DATA' section shows 'With data' and 'Without data' tabs. The 'With data' tab is active, displaying a table with columns: Host, Name, Last check, Last value, Change, and Tags. The table contains three rows of data for the 'serverb' host, showing load average (1m avg), load average (5m avg), and load average (15m avg) for the 'component: cpu' tag.

Host	Name	Last check	Last value	Change	Tags
serverb	Load average (1m avg)	16s	3.9297	+0.1255	component: cpu
serverb	Load average (5m avg)	15s	2.2456	+0.3926	component: cpu
serverb	Load average (15m avg)	17s	0.9209	+0.2085	component: cpu

Task 13 – Stop the CPU Load and Verify Recovery

After the monitoring test was completed, the CPU-consuming processes were stopped.

On servera

```
pskill yes
```

On serverb

kill yes

After allowing Zabbix to collect new values, the load values decreased.

For the **Zabbix server**, the 1-minute load decreased to approximately:

0.006836

For **serverb**, the 1-minute load decreased to approximately:

0.0669

This demonstrated that Zabbix continued collecting data after the CPU stress test and reflected the recovery of both servers.

Screenshot Output

Subfilter affects only filtered data

HOSTS
Zabbix server 3

TAGS
component 3

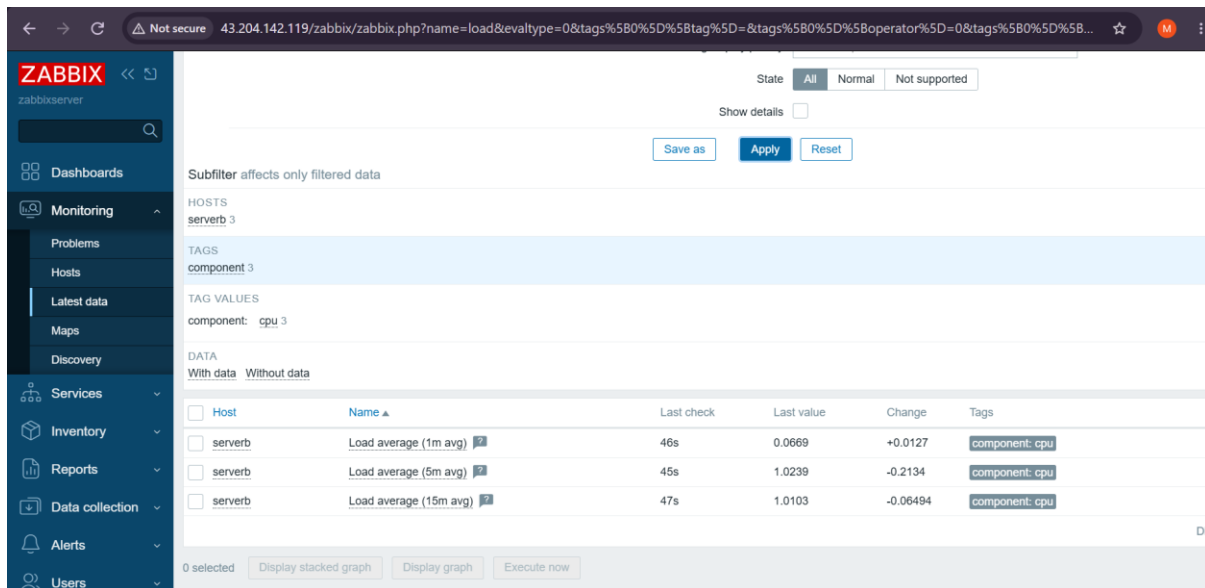
TAG VALUES
component: cpu 3

DATA
With data Without data

☐	Host	Name ▲	Last check	Last value	Change	Tags
☐	Zabbix server	Load average (1m avg) 📊	51s	0.006836	-0.01709	component: cpu
☐	Zabbix server	Load average (5m avg) 📊	43s	0.9087	-0.2051	component: cpu
☐	Zabbix server	Load average (15m avg) 📊	41s	1.0498	-0.07324	component: cpu

0 selected Display stacked graph Display graph Execute now

Zabbix 7.4.14, © 2001–2026, Zabbix SIA



Result

The Zabbix monitoring practical was completed successfully.

- Zabbix Server 7.4 was installed on servera.
- MariaDB was configured as the Zabbix database.
- The Zabbix web interface was configured successfully.
- Zabbix Agent 2 was installed and configured on servera and serverb.
- serverb was added as a monitored host.
- The Zabbix server itself was also monitored.
- Communication through Zabbix Agent port 10050 was verified.
- agent.ping successfully returned 1.
- CPU/load values were successfully collected from both servers.
- CPU stress was generated on both servers.
- Increased CPU load was visible in Zabbix.
- The stress processes were stopped.
- Zabbix reflected the reduction in CPU load after recovery.

Conclusion

The Zabbix monitoring environment was successfully configured using two RHEL EC2 instances. servera was configured as the Zabbix Server and also monitored using Zabbix Agent 2, while serverb was configured as a monitored client using Zabbix Agent 2. Communication between the Zabbix server and agents was verified, and real-time CPU/load

monitoring was tested by generating and removing system load. The successful collection and recovery of the load values confirmed that the Zabbix monitoring setup was functioning correctly.